How an animations break down into pieces and low level logic behind it:

0. animation
1. frame
2. lyr_frame
3. tlc5940 IC level

An animation is a series of 8x8x8 cube RGB frames.

A frame is a struct containing 8 layer frame structs.

A layer frame is an 8x8 horizontal slice of the cube where layer0 is ground level and layer7 is the top.

The cube multiplexes these 8 layers, using persistence of vision to make it seem like all 8 layers are illuminated at once.

A layer frame is a struct of 64 element uint16_t arrays to to represent a given layer frame's red, green, and blue values at a given time.

While the layer frame structs are 16bit, the tlc5940 pwm driver uses 12bit intensity numbers (really capture compare values for timers which per channel toggle the leds off when the count goes above that value).

There is a function which converts the layer frame struct data into bit shifted 12bit aligned pwm values to shift into the daisy chained 12x tlc5940 ICs, each of which drive 16 individual LEDs of the 192 common anode LEDs comprising the cube.

Each PWM driver LED channel is set to sink 20mA max by a set resistor on the PCBA, thus 3.84A max when all LEDs are on at full brightness.

The tlc5940 implementation has no knowledge of this higher-level information regarding the animations, just the PWM bit stream it is fed, plus some logic to control some key signals, like the timer clock (GSCLK), the XLAT pin to latch the shifted data after it is shifted in via spi before the gsclk pin begins, the clear pin, which must be triggered once the timer reaches 4095, etc.

I should show some logic analyzer captures of the tlc5940 signals happening real-time and the corresponding functions in firmware.

Ideas for three different animation styles / modes. Cube in its existing state mainly has algorithmic animations with the framework set for video mode, except still needing an ext flash reader, like an SD card perhaps. That and a piece of PC software to generate the animations which I'd write.

- Video Mode
    - Cube runs through pre-scripted pattern sequences
- Interactive Mode
    - Cube takes inputs from peripherals to generate patterns
- Debug Mode
    - Allows user to toggle LEDs on and off manually for HW testing

Additional animation notes:

The cube's existing firmware / flash size is full / near random full of algorithmic animations.

Adding an external storage reading function is what would be required in the future to enable software defined animations to be loaded onto the cube.

A bool mask is a collection of 64 element bools which I was using to qickly generate and toggle between different layer frames by multiplying color values by them to quickly build animations.

Where I left off, I was using the cube test board which is a single 8x8 layer, not the entire cube. That way I didn't have to stress about potentially damaging it during development before I was ready.

It is easy to still test the full multiplexing logic by modifying how the power FETs are wired to the test board's common power rail.

menu_system.c
- mpr121.c
- lcd_20x4_i2c.c

I thought it would be fun to use a capacitive touch scroll wheel and keypad to control the cube with an LCD of course.

I successfully implemented libraries for the scroll wheel such that spinning the wheel triggers various touch detected activities in a sequence such that the firmware can detect if scrolling one direction vs the other. That can result in scrolling up or down on the LCD, as well as the center button being a select key.

While this works, simple push buttons would have better, but maybe not as cool looking.
My main complaint right now is that the capacitive touch is a little sensitive to skin moisture Excessively dry or sweaty hands might not trigger it as well as an otherwise normal finger.

Aside from this, the design does not need both a scroll wheel and a touch pad.
The touch pad would have been more straightforward, but I did the scroll wheel instead.